

CARTES ANIMÉES

Manuel d'installation et de déploiement

Installer, initialiser et exploiter l'application en développement
et en production

Projet	Cartes Animées – application web éducative pour enfants malentendants
Cadre	Projet de fin d'études – Mastère Expert en Systèmes d'Information (ESI2), Institut Limayrac
Auteur	Jean Lautraite
Tuteur technique	Stéphane Blusson
Tutrice pédagogique	Géraldine Boutan
Version	1.0
Date	29 août 2026

Table des matières

1. Vue d'ensemble.....	3
1.1 Prérequis.....	3
2. Installation locale.....	4
2.1 Récupérer le projet et installer les dépendances.....	4
2.2 Configurer l'environnement (.env).....	4
2.3 Base de données, stockage et assets.....	5
3. Configuration Stripe.....	5
3.1 Clés d'API.....	5
3.2 Webhooks en développement.....	5
3.3 Événements traités.....	6
3.4 Cartes de test.....	6
4. Initialisation de l'application.....	6
4.1 Données de démonstration (--seed).....	7
5. Démarrage en développement.....	7
6. Déploiement en production.....	8
6.1 Environnement cible.....	8
6.2 Mise en place.....	8
6.3 Hôte virtuel nginx.....	8
6.4 Worker de queue (obligatoire).....	9
6.5 Planificateur (cron).....	9
6.6 Webhook Stripe en production.....	9
6.7 Vérifications post-déploiement.....	9
7. Dépannage.....	10

1. Vue d'ensemble

Cartes Animées est une application **Laravel 12** unique qui sert trois surfaces : l'API REST (/api), les panels d'administration Filament (/admin et /therapist) et l'application web Vue 3 compilée par Vite (toutes les autres routes). Il n'y a donc **qu'un seul déploiement** : pas de serveur front distinct, pas de configuration CORS.

Les dépendances externes sont : une base MySQL, l'API Stripe (abonnements et paiements) et un serveur SMTP pour les e-mails (Mailpit en développement).

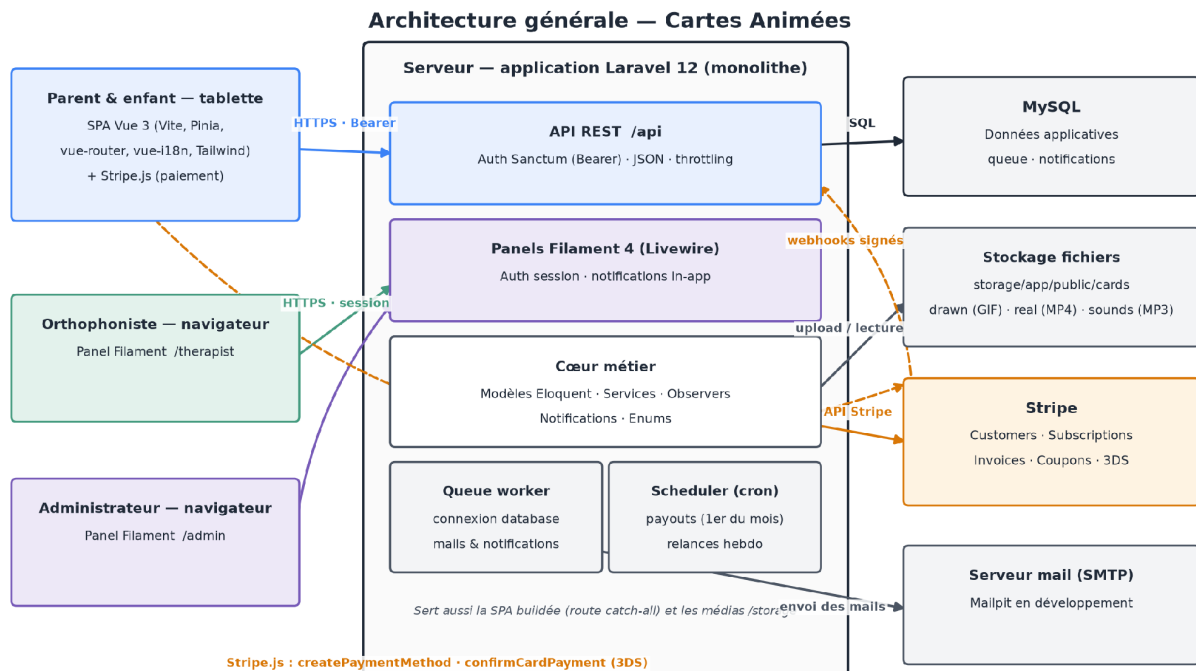


Figure 1 – Architecture générale de l'application.

1.1 Prérequis

Composant	Version / remarque
PHP	8.2 minimum (8.4 recommandé), avec les extensions Laravel usuelles (mbstring, intl, pdo_mysql, gd, zip, curl...)
Composer	2.x
Node.js + npm	Node 20 ou supérieur (build des assets Vite)
MySQL	8.x (une base dédiée, p. ex. cartes_animees)
Git	clonage du dépôt
Compte Stripe	mode test (sandbox) pour le développement et la démonstration
Stripe CLI	développement uniquement – relaie les webhooks vers le poste local
Laravel Herd	recommandé en local (sert le site sur http://cartes-animees.test) ; à défaut, php artisan serve

2. Installation locale

2.1 Récupérer le projet et installer les dépendances

```
git clone https://github.com/LautrJ/cartes-animees.git
cd cartes-animees
composer install
npm install
```

2.2 Configurer l'environnement (.env)

```
cp .env.example .env
php artisan key:generate
```

Variable	Valeur (exemple local)	Rôle
APP_NAME	Cartes Animées	Nom de l'application (e-mails, titres)
APP_ENV	local	local en développement, production en prod
APP_URL	http://cartes-animees.test	URL racine – sert aussi à construire les URL des médias /storage
FRONTEND_URL	http://cartes-animees.test	URL de la SPA – utilisée dans les liens de réinitialisation de mot de passe
FILAMENT_IMPERSONATE_REDIRECT	/impersonate/bridge	Pont d'impersonation admin → application parent
APP_LOCALE	fr	Langue de l'application
DB_CONNECTION	mysql	Base de données
DB_HOST / DB_PORT	127.0.0.1 / 3306	
DB_DATABASE	cartes_animees	Base à créer au préalable
DB_USERNAME / DB_PASSWORD	root / (vide)	Selon l'installation MySQL
QUEUE_CONNECTION	database	Obligatoire : mails et notifications passent par la queue
MAIL_MAILER	smtp	Mailpit en local
MAIL_HOST / MAIL_PORT	127.0.0.1 / 1025	Port SMTP de Mailpit
MAIL_FROM_ADDRESS	noreply@cartes-animees.fr	Expéditeur des e-mails
STRIPE_KEY	pk_test_...	Clé publique Stripe
VITE_STRIPE_KEY	pk_test_... (même valeur)	Clé publique exposée à la SPA (Stripe.js) – recompiler

		les assets après modification
STRIPE_SECRET	sk_test_...	Clé secrète Stripe (ne jamais l'exposer)
STRIPE_WEBHOOK_SECRET	whsec_...	Secret de signature des webhooks (voir section 3)
STRIPE_PRODUCT_ID	prod_...	Rempli automatiquement par app:init – ne pas saisir à la main

i Les variables VITE_* sont figées dans le bundle JavaScript au moment du build : après toute modification, relancer `npm run build` (ou le serveur `npm run dev`).

2.3 Base de données, stockage et assets

```
# Créer la base (si besoin)
mysql -u root -e "CREATE DATABASE cartes_animees CHARACTER SET utf8mb4"

# Lien symbolique public/storage -> storage/app/public
php artisan storage:link

# Compiler les assets (SPA Vue + thèmes Filament)
npm run build
```

⚠ Sans `php artisan storage:link`, les médias des cartes (GIF, vidéos, sons) et les miniatures renverront des erreurs 404 : ils sont servis depuis `/storage/cards/...`.

Les migrations peuvent être lancées avec `php artisan migrate`, mais l'initialisation complète recommandée passe par `app:init` (section 4), qui exécute lui-même `migrate:fresh`.

3. Configuration Stripe

3.1 Clés d'API

1. Créer un compte sur stripe.com et rester en **mode test** (sandbox) pour le développement et la démonstration.
2. Dans le dashboard Stripe : « Développeurs → Clés API », copier la clé publique (`pk_test_...`) dans `STRIPE_KEY` et `VITE_STRIPE_KEY`, et la clé secrète (`sk_test_...`) dans `STRIPE_SECRET`.
3. Le produit et le prix Stripe sont créés automatiquement par `app:init` (section 4) : `STRIPE_PRODUCT_ID` est alors écrit dans le `.env`.

3.2 Webhooks en développement

Stripe notifie l'application des événements de facturation via des webhooks. En local, la Stripe CLI relaie ces événements vers le poste de développement :

```
stripe login
stripe listen --forward-to http://cartes-animees.test/api/stripe/webhook
```

La commande affiche un secret de signature `whsec_...` : le reporter dans `STRIPE_WEBHOOK_SECRET`. La CLI doit rester ouverte pendant les tests de paiement.

3.3 Événements traités

Le gestionnaire de webhooks vérifie la signature de chaque événement puis traite les cas suivants :

Événement Stripe	Traitement
<code>invoice.payment_succeeded</code>	Enregistre la facture (payée), passe l'abonnement en active, met à jour la période, e-mail de confirmation au parent
<code>invoice.payment_failed</code>	Enregistre la facture (ouverte). Premier paiement : l'abonnement reste incomplete (reprise possible). Renouvellement : passage en <code>past_due</code> + e-mail d'alerte au parent
<code>customer.subscription.updated</code>	Synchronise le statut et les dates de période
<code>customer.subscription.deleted</code>	Passe l'abonnement en canceled
<code>invoice.finalized</code>	Enregistre la facture ouverte avec son PDF

Les autres événements reçus sont ignorés silencieusement ; les événements inconnus sont journalisés. Toute l'activité Stripe est tracée dans un canal de log dédié : `storage/logs/stripe-*.log` (rotation quotidienne, 30 jours).

3.4 Cartes de test

Carte	Comportement
4242 4242 4242 4242	Paiement accepté sans 3D Secure
4000 0027 6000 3184	Déclenche l'authentification 3D Secure
4000 0000 0000 0002	Carte refusée (decline générique)
4000 0000 0000 9995	Refusée pour fonds insuffisants

Date d'expiration future et CVC quelconques. D'autres scénarios (échec de renouvellement...) peuvent être simulés :

```
stripe trigger invoice.payment_failed
stripe trigger invoice.payment_succeeded
stripe trigger customer.subscription.deleted
```

4. Initialisation de l'application

La commande `php artisan app:init` prépare une application prête à l'emploi. Elle est **interactive** et **destructive** : elle commence par demander confirmation, car elle remet la base à zéro.

1. Confirmation – la commande efface toutes les données existantes (`migrate:fresh`).
2. Nettoyage du sandbox Stripe (archivage des données de test précédentes) puis création du produit Stripe ; `STRIPE_PRODUCT_ID` est écrit dans le `.env`.

3. Saisie du **prix mensuel de l'abonnement** (défaut : 7,99 €).
4. Saisie du **mot de passe administrateur** – le compte `admin@cartes-animees.test` est créé.
5. Saisie du **taux de commission** des orthophonistes (défaut : 2,00 €/patient/mois).
6. Création du prix Stripe et des premières entrées d'historique (prix et taux).

```
# Initialisation seule (production, démo vierge)
php artisan app:init

# Initialisation + données de démonstration (développement)
php artisan app:init --seed
```

4.1 Données de démonstration (--seed)

L'option `--seed` injecte un jeu de données réaliste :

- 3 orthophonistes avec leurs coordonnées bancaires ;
- 7 parents (dont 2 inactifs depuis plus de deux semaines), 2 enfants chacun, affiliés aléatoirement aux orthophonistes ;
- des abonnements **réellement créés dans le sandbox Stripe** : 7 actifs, 1 gratuit, 1 en échec de paiement (`past_due`), 1 annulé – avec leurs factures ;
- 15 cartes validées (admin), des cartes et une série en attente de validation ;
- 3 séries de base + 5 séries classiques, composées de cartes aléatoires ;
- la progression des enfants (séries débloquées/complétées) et 3 mois de virements calculés.

⚠ Le seed effectue de **vrais appels à l'API Stripe sandbox** : il nécessite des clés valides, un peu de temps, et le **worker de queue doit tourner** pour que les notifications et e-mails soient traités.

5. Démarrage en développement

```
composer dev
```

Cette commande unique lance en parallèle :

Processus	Rôle
<code>php artisan serve</code>	Serveur HTTP de développement (inutile avec Herd, qui sert déjà le site)
<code>php artisan queue:listen</code>	Worker de queue – obligatoire pour les e-mails et notifications
<code>npm run dev</code>	Vite : compilation à la volée et rechargement à chaud de la SPA

Dans un second terminal, pour les paiements :

```
stripe listen --forward-to http://cartes-animees.test/api/stripe/webhook
```

Les e-mails de développement sont consultables dans Mailpit : `http://localhost:8025`.

Surface	URL locale
Application web (parents / enfants)	http://cartes-animees.test/
Panel orthophoniste	http://cartes-animees.test/therapist
Panel administrateur	http://cartes-animees.test/admin
Mailpit (e-mails)	http://localhost:8025

6. Déploiement en production

6.1 Environnement cible

Le déploiement de référence est un serveur Linux (Ubuntu/Debian) avec nginx + PHP-FPM, MySQL et un certificat TLS (le HTTPS est indispensable : Stripe l'exige pour les paiements). Node.js n'est nécessaire que le temps du build des assets.

6.2 Mise en place

```
cd /var/www
git clone https://github.com/LautrJ/cartes-animees.git
cd cartes-animees

composer install --no-dev --optimize-autoloader
npm ci && npm run build

cp .env.example .env          # puis compléter (voir 2.2, valeurs de production)
php artisan key:generate
php artisan storage:link
php artisan migrate --force
php artisan app:init          # sans --seed en production

# Droits d'écriture pour PHP-FPM
chown -R www-data:www-data storage bootstrap/cache

# Caches de production
php artisan config:cache && php artisan route:cache && php artisan view:cache
```

Différences du .env de production : APP_ENV=production, APP_DEBUG=false, APP_URL et FRONTEND_URL en https://..., identifiants MySQL dédiés, SMTP réel, et clés Stripe du mode choisi (le projet reste en sandbox pour la démonstration du PFE).

6.3 Hôte virtuel nginx

```
server {
    listen 443 ssl http2;
    server_name cartes-animees.fr;
    root /var/www/cartes-animees/public;
    index index.php;

    ssl_certificate      /etc/letsencrypt/live/cartes-animees.fr/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/cartes-animees.fr/privkey.pem;

    client_max_body_size 50M;    # téléversement des vidéos par les orthophonistes
}
```

```
location / {
    try_files $uri $uri/ /index.php?$query_string;
}

location ~ \.php$ {
    include snippets/fastcgi-php.conf;
    fastcgi_pass unix:/run/php/php8.4-fpm.sock;
}
}
```

i Aligner aussi PHP sur la taille des médias : `upload_max_filesize = 50M` et `post_max_size = 50M` dans le `php.ini` de PHP-FPM.

6.4 Worker de queue (obligatoire)

Les e-mails et notifications sont mis en file (connexion database) : sans worker permanent, rien ne part. Service `systemd` recommandé :

```
# /etc/systemd/system/cartes-animees-queue.service
[Unit]
Description=Cartes Animees - queue worker
After=network.target mysql.service

[Service]
User=www-data
Restart=always
ExecStart=/usr/bin/php /var/www/cartes-animees/artisan queue:work --sleep=3 --tries=3

[Install]
WantedBy=multi-user.target

systemctl enable --now cartes-animees-queue
```

6.5 Planificateur (cron)

Deux traitements sont planifiés dans l'application : le calcul des virements orthophonistes (le 1^{er} du mois à 6 h) et la relance des parents inactifs (le lundi à 9 h). Une seule ligne cron les déclenche :

```
* * * * * cd /var/www/cartes-animees && php artisan schedule:run >> /dev/null 2>&1
```

6.6 Webhook Stripe en production

1. Dashboard Stripe → « Développeurs → Webhooks » → « Ajouter un endpoint » : <https://cartes-animees.fr/api/stripe/webhook>.
2. Sélectionner les événements de la section 3.3 (au minimum `invoice.payment_succeeded`, `invoice.payment_failed`, `invoice.finalized`, `customer.subscription.updated`, `customer.subscription.deleted`).
3. Copier le secret de signature de l'endpoint dans `STRIPE_WEBHOOK_SECRET` puis relancer `php artisan config:cache`.

6.7 Vérifications post-déploiement

Vérification	Attendu
Ouvrir / , /therapist et /admin	Les trois surfaces répondent en HTTPS
Créer un compte parent de test	Inscription OK, e-mail de bienvenue reçu
Créer un enfant	Les séries de base apparaissent immédiatement
Souscrire avec la carte 4242...	Abonnement « Abonné » après paiement ; facture visible dans /admin
Consulter storage/logs/stripe-*.log	Les événements webhook y sont tracés sans erreur de signature
systemctl status cartes-animees-queue	Worker actif ; les e-mails partent
php artisan schedule:list	Les deux tâches planifiées sont listées

7. Dépannage

Symptôme	Cause probable	Solution
Images/GIF/vidéos en 404	Lien symbolique absent	<code>php artisan storage:link</code>
Aucun e-mail ni notification	Worker de queue arrêté	Lancer <code>php artisan queue:work</code> (ou vérifier le service <code>systemd</code>)
Webhook en erreur 400	Secret de signature incorrect	Vérifier <code>STRIPE_WEBHOOK_SECRET</code> (celui du stripe listen local ou de l'endpoint de production), puis <code>config:cache</code>
Paieement validé côté Stripe mais abonnement toujours « En attente »	Webhooks non reçus	En local : stripe listen doit tourner. En production : vérifier l'endpoint et ses événements
Page blanche / erreur Vite manifest	Assets non compilés	<code>npm run build</code>
Champ carte bancaire vide sur /subscribe	<code>VITE_STRIPE_KEY</code> absente du build	Renseigner la variable puis relancer <code>npm run build</code>
419 / problème de session sur les panels	<code>APP_URL</code> incorrect ou cache de config périmé	Corriger <code>APP_URL</code> puis <code>php artisan config:cache</code>
<code>app:init</code> échoue sur Stripe	Clés absentes ou invalides	Vérifier <code>STRIPE_KEY</code> / <code>STRIPE_SECRET</code> (mode test) dans le <code>.env</code>